

Python code :

Set up

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme()
```

arrays + math
numerically solve ODE
make plots
make plots look
good

SHO

```
t = np.linspace(0, 15, 1000)
omega_sq = 1
y = [0, 2] # y[0] = x and y[1] = v
```

```
def harmonic(t, y):
    solution = [y[1], -omega_sq * y[0]]
    return solution
```

```
sho = solve_ivp(harmonic, [0, 1000], y0=y, t_eval=t)
```

This solves the SHO equation of motion :

$$\ddot{x} = -\omega^2 x$$

To solve it numerically, it's rewritten as two coupled first-order equations using state vector $y = (x, v)$:

$$\dot{x} = v \quad (\text{that's } y[1])$$

$$\dot{v} = -\omega^2 x \quad (\text{that's } -\omega^2 * y[0])$$

$y = [0, 2]$ is the initial condition: position 0, velocity 2. `solve_ivp` integrates from $t=0$ to $t=1000$, but `t_eval=t` tells it to only report the solution at the 1000 points between 0 and 15. `sho.y[0]` afterward holds the position values over time.

```
plt.plot(t, sho.y[0])  
plt.ylabel("Position")  
plt.xlabel("Time")  
plt.title('SHO', fontsize = 20)
```

This plots position vs time

Damped Oscillator

```
t = np.linspace(0, 15, 1000)  
y = [0, 1]
```

```
gamma = 1  
omega_sqr = 100
```

```
def sho(t, y):  
    solution = (y[1], (-gamma*y[1] - omega_sqr*y[0]))  
    return solution
```

```
solution = solve_ivp(sho, [0, 1000], y0=y, t_eval=t)
```

```
plt.plot(t, solution.y[0])  
plt.ylabel("Position")  
plt.xlabel("Time")  
plt.title('Damped Oscillator', fontsize = 20)
```

Same idea, but now with a damping term:

$$\ddot{x} = -\gamma \dot{x} - \omega^2 x$$

The $-\gamma \dot{x}$ term is friction/damping proportional to velocity. With $\omega_{\text{spr}} = 100$, this oscillates much faster than the SHO.